

To actually create a window we need to call the API function **CreateWindow()**. This function requires several parameters starting with the window class. The second parameter indicates the text that is going to appear on the button surface. The third parameter specifies the window style. **WS_OVERLAPPEDWINDOW** is a commonly used style. The next four parameters specify the window's initial position and size—the x and y screen coordinates of the window's top left corner and the window's width and height in pixels. The next three parameters specify the handles to the parent window, the menu and the application instance respectively. The last parameter is the pointer to the window-creation data.

We can easily devote a section of this book to **CreateWindow()** and its parameters. But don't get scared of it. Nobody is supposed to remember all the parameters, their meaning and their order. You can always use MSDN (Microsoft Developer Network) help to understand the minute details of each parameter. This help is available as part of VC++ 6.0 product. It is also available on the net at <http://www.msdn.microsoft.com/library>.

Note that **CreateWindow()** merely creates the window in memory. We still are to display it on the screen. This can be done using the **ShowWindow()** API function. **CreateWindow()** returns handle of the created window. Our program uses this handle to refer to the window while calling **ShowWindow()**. The second parameter passed to **ShowWindow()** signifies whether the window would appear minimized, maximized or normal. If the value of this parameter is **SW_SHOWNORMAL** we get a normal sized window, if it is **SW_SHOWMINIMIZED** we get a minimized window and if it is **SW_SHOWMAXIMIZED** we get a maximized window. We have passed **nCmdShow** as the second parameter. This variable contains **SW_SHOWNORMAL** by default. Hence our program displays a normal sized window.

The `WS_OVERLAPPEDWINDOW` style is a collection of the following styles:

```
WS_OVERLAPPED | WS_CAPTION | WS_SYSMENU | WS_THICKFRAME |  
WS_MINIMIZEBOX | WS_MAXIMIZEBOX
```

As you can make out from these macros they essentially control the look and feel of the window being created. All these macros are **#defined** in the 'Windows.h' header file.

On executing this program a window and a message box appears on the screen as shown in the Figure 17.2. The window and the message box disappear as soon as we click on OK. This is because on doing so execution of **WinMain()** comes to an end and moreover we have made no provision to interact with the window.

You can try to remove the call to **MessageBox()** and see the result. You would observe that no sooner does the window appear it disappears. Thus a call to **MessageBox()** serves the similar purpose as **getch()** does in sequential programming.

More Windows

Now that we know how to create a window let us create several windows on the screen. The program to do this is given below.

```
#include <windows.h>  
  
int _stdcall WinMain ( HINSTANCE hInstance, HINSTANCE hPrevInstance,  
                      LPSTR lpszCmdLine, int nCmdShow )  
{  
    HWND h[10] ;  
    int x ;  
  
    for ( x = 0 ; x <= 9 ; x++ )  
    {
```

```
h[x] = CreateWindow ( "BUTTON", "Press Me",  
                    WS_OVERLAPPEDWINDOW, x * 20,  
                    x * 20, 150, 100, 0, 0, i, 0 );  
ShowWindow ( h[x], 1 );  
}  
  
MessageBox ( 0, "Hi!", "Waiting", 0 );  
return 0 ;  
}
```

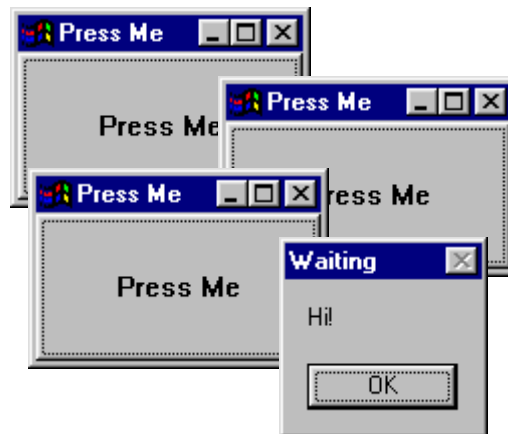


Figure 17.3

Note that each window created in this program is assigned a different handle. You may experiment a bit by changing the name of the window class to EDIT and see the result.

A Real-World Window

Suppose we wish to create a window and draw a few shapes in it. For creating such a window there is no standard window class available. Hence we would have to create our own window class, register it with Windows OS and then create a window on the basis of it. Instead of straightway jumping to a program that draws

shapes in a window let us first write a program that creates a window using our window class and lets us interact with it. Here is the program...

```
#include <windows.h>
#include "helper.h"

void OnDestroy ( HWND ) ;

int __stdcall WinMain ( HINSTANCE hInstance, HINSTANCE hPrevInstance,
                      LPSTR lpszCmdline, int nCmdShow )
{
    MSG m ;

    /* perform application initialization */
    InitInstance ( hInstance, nCmdShow, "title" ) ;

    /* message loop */
    while ( GetMessage ( &m, 0, 0, 0 ) )
        DispatchMessage ( &m ) ;

    return 0 ;
}

LRESULT CALLBACK WndProc ( HWND hWnd, UINT message,
                          WPARAM wParam, LPARAM lParam )
{
    switch ( message )
    {
        case WM_DESTROY :
            OnDestroy ( hWnd ) ;
            break ;
        default :
            return DefWindowProc ( hWnd, message, wParam, lParam ) ;
    }
    return 0 ;
}
```

```
void OnDestroy ( HWND hWnd )  
{  
    PostQuitMessage ( 0 );  
}
```

On execution of this program the window shown in Figure 17.4 appears on the screen. We can use minimize and the maximize button in its title bar to minimize and maximize the window. We can stretch its size by dragging its boundaries. Finally, we can close the window by clicking on the close window button in the title bar.

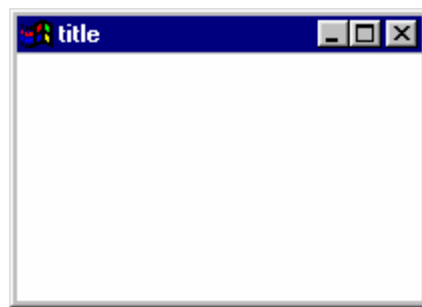


Figure 17.4

Let us now try to understand this program step by step.

Creation and Displaying of Window

Creating and displaying a window on the screen is a 4-step process. These steps are:

- (a) Creation of a window class.
- (b) Registering the window class with the OS.
- (c) Creation of a window based on the registered class.
- (d) Displaying the window on the screen.

Creation of a window class involves setting up of elements of a structure called **WNDCLASSEX**. This structure contains several

elements. They govern the properties of the window. Registration of a window class, creation of a window and displaying of a window involves calling of API functions **RegisterClassEx()**, **CreateWindow()** and **ShowWindow()** respectively. Since all the 4 steps mentioned above would be required in almost every program in this chapter I have written this code in a user-defined function called **InitInstance()** in the file 'helper.h'.

Though writing code in a header file goes against the convention I have still done so to achieve simplicity. The complete listing of 'helper.h' file is available in Appendix F. Alternatively you can download it from the following link:

www.kicit.com/books/letusc/sourcecode/helper.h

As expected **WinMain()** starts off by calling the function **InitInstance()** present in 'helper.h' file. This file has been **#included** at the beginning of the program. Remember to copy this file to your project directory—the directory in which you are going to create this program.

Once the window has been created and displayed let us see how we can interact with it.

Interaction with Window

As and when the user interacts with the window—by stretching its boundaries or clicking the buttons in the title bar, etc. a suitable message is posted into the message queue of our application. Our application should now pick them up from the message queue and process them.

A message contains a message id and some other additional information about the message. For example, a mouse click message would contain additional information like handle to the window with which the user has interacted, the coordinates of

mouse cursor and the status of mouse buttons. Since it is difficult to memorize the message ids they have been suitably **#defined** in 'windows.h'. The message id and the additional information are stored in a structure called MSG.

In **WinMain()** this MSG structure is retrieved from the message queue by calling the API function **GetMessage()**. The first parameter passed to this function is the address of the **MSG** structure variable. **GetMessage()** would pick the message info from the message queue and place it in the structure variable passed to it. Don't bother about the other parameters right now.

After picking up the message from the message queue we need to process it. This is done by calling the **DispatchMessage()** API function. This function does several activities. These are as follows:

- (a) From the **MSG** structure that we pass to it, **DispatchMessage()** extracts the handle of the window for which this message is meant for.
- (b) From the handle it figures out the window class based on which the window has been created.
- (c) From the window class structure it obtains the address of a function called **WndProc()** (short for window procedure). Well I didn't tell you earlier that in **InitInstance()** while filling the **WNDCLASSEX** structure one of the elements has been set up with the address of a user-defined function called **WndProc()**.
- (d) Using this address it calls the function **WndProc()**.

Since several messages get posted into the message queue picking of the message and processing it should be done repeatedly. Hence calls to **GetMessage()** and **DispatchMessage()** have been made in a **while** loop in **WinMain()**. When **GetMessage()** encounters a message with id **WM_QUIT** it returns a **0**. Now the control comes out of the loop and **WinMain()** comes to an end.

Reacting to Messages

As we saw in the previous section, for every message picked up from the message queue the control is transferred to the **WndProc()** function. This function is shown below:

```
LRESULT CALLBACK WndProc ( HWND hWnd, UINT message,  
                           WPARAM wParam, LPARAM lParam )
```

This function always receives four parameters. The first parameter is the handle to the window for which the message has been received. The second parameter is the message id, whereas, the third and fourth parameters contain additional information about the message.

LRESULT is a **typedef** of a **long int** and represents the return value of this function. **CALLBACK** is a **typedef** of **__stdcall**. This **typedef** has been done in 'windows.h'. **CALLBACK** indicates that the **WndProc** function has been registered with Windows (through **WNDCLASSEX** structure in **InitInstance()**) with an intention that Windows would call this back (through **DispatchMessage()** function).

In the **WndProc()** function we have checked the message id using a **switch**. If the id is **WM_DESTROY** then we have called the function **OnDestroy()**. This message is posted to the message queue when the user clicks on the 'Close Window' button in the title bar. In **OnDestroy()** function we have called the API function **PostQuitMessage()**. This function posts a **WM_QUIT** message into the message queue. As we saw earlier, when this message is picked up the message loop and **WinMain()** is terminated.

For all messages other than **WM_DESTROY** the control lands in the **default** clause of **switch**. Here we have simply made a call to **DefWindowProc()** API function. This function does the default

processing of the message that we have decided not to tackle. The default processing for different message would be different. For example on double clicking the title bar **DefWindowProc()** maximizes the window.

Actually speaking when we close the window a **WM_CLOSE** message is posted into the message queue. Since we have not handled this message the **DefWindowProc()** function gets called to tackle this message. The **DefWindowProc()** function destroys the window and places a **WM_DESTROY** message in the message queue. As discussed earlier, in **WndProc()** we have made the provision to terminate the application on encountering **WM_DESTROY**.

That brings us to the end of a lonnnngggg explanation! You can now heave a sigh of relief. I would urge you to go through the above explanation till the time you are absolutely sure that you have understood every detail of it. A very clear understanding of it would help you make a good Windows programmer. For your convenience I have given a flowchart of the entire working in Figure 17.5.

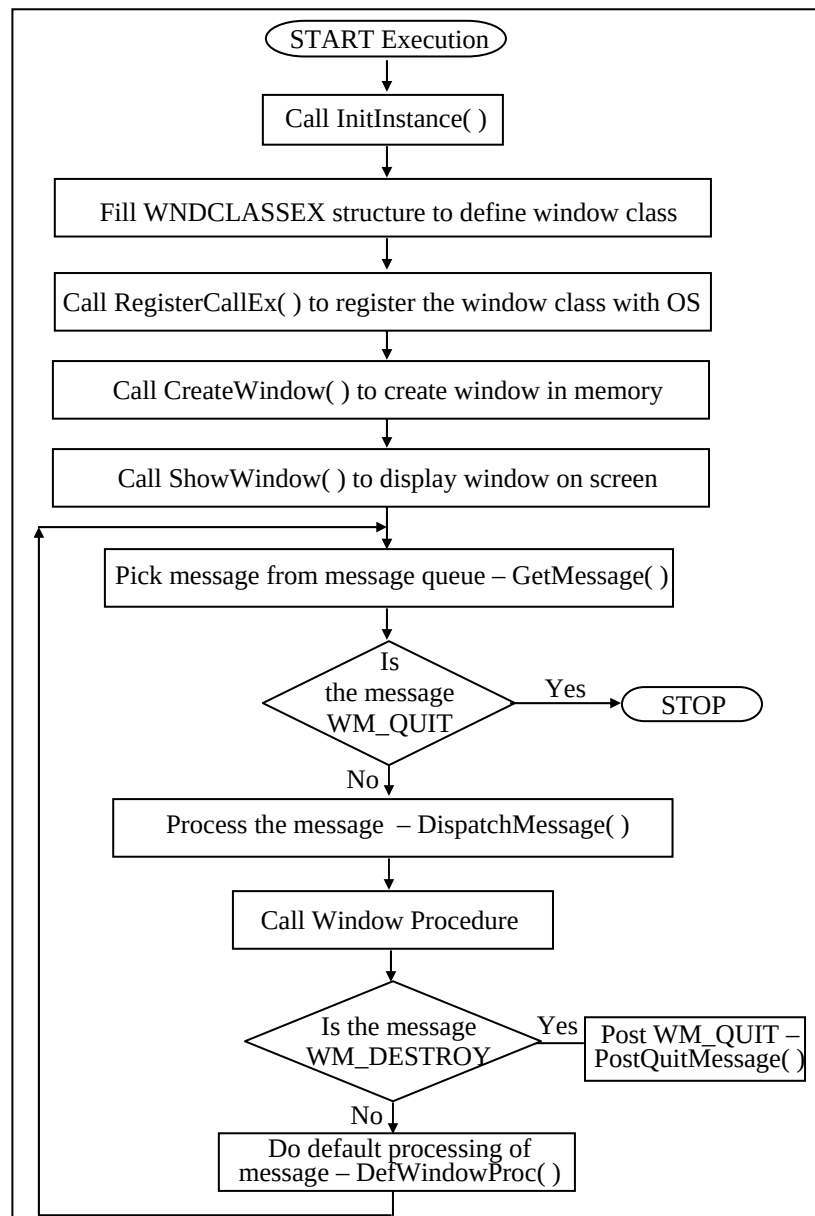


Figure 17.5

Program Instances

Windows allows you to run more than one copy of a program at a time. This is handy for cutting and pasting between two copies of Notepad or when running more than one terminal session with a terminal emulator program. Each running copy of a program is called a 'program instance'.

Windows performs an interesting memory optimization trick. It shares a single copy of the program's code between all running instances. For example, if you get three instances of Notepad running, there will only be one copy of Notepad's code in memory. All three instances share the same code, but will have separate memory areas to hold the text data being edited. The difference between handling of the code and the data is logical, as each instance of Notepad might edit a different file, so the data must be unique to each instance. The program logic to edit the files is the same for every instance, so there is no reason why a single copy of Notepad's code cannot be shared.

Summary

- (a) A message box can be displayed by calling the **MessageBox()** API function.
- (b) Message boxes are often used to ascertain the flow of a program.
- (c) Appearance of a message box can be customized.
- (d) The **CreateWindow()** API function creates the window in memory.
- (e) The window that is created in memory is displayed using the **ShowWindow()** API function.
- (f) A 'window class' specifies various properties of the window that we are creating.
- (g) The header file 'Windows.h' contains declaration of several macros used in Windows programming.

- (h) When the user clicks in a window, or moves mouse pointer on the window, messages are generated and posted in the application message queue.
- (i) A message contains the message id and additional information about the message.
- (j) The **GetMessage()-DispatchMessage()** loop breaks when **GetMessage()** encounters the **WM_QUIT** message.
- (k) If we don't handle a message received by our application then the **DefWindowProc()** function is called to do the default processing.

Exercise

[A] State True or False:

- (a) **MessageBox()** is an API function.
- (b) Calling the **MessageBox()** function displays the specified string in console window.
- (c) The **CreateWindow()** function creates and displays the window on the screen.
- (d) The **ShowWindow()** function can display only the maximized window.
- (e) Every window has to be created using pre-registered window class.
- (f) Window classes are similar to classes in C++.
- (g) We can use the pre-defined window classes but cannot create our own.
- (h) The style **WS_OVERLAPPED | WS_CAPTION | WS_MINIMIZEBOX** will create a window with caption bar and minimize box only.
- (i) To be able to interact with a window it is necessary to implement the message loop.

[B] Answer the following:

- (a) Outline the steps that a typical Windows program follows during execution.

- (b) Run any Windows based program and see whether you can identify all the elements of the application window.
- (c) How would you minimize a window programmatically?
- (d) What would happen if we do not place WM_QUIT message in the message queue when the user tries to close the window.
- (e) Explain the need of **RegisterClassEx()** function.
- (f) What is the difference between **GetMessage()** and **DispatchMessage()** function?
- (g) Write a program, which receives an integer as a command line argument, creates a button window, and based on the value of the integer displays button window as maximized / minimized / normal.
- (h) Try to display a window with different combinations of window styles and observe the results.

18 Graphics Under Windows

- Graphics as of Now
- Device Independent Drawing
- Hello Windows
- Drawing Shapes
- Types of Pens
- Types of Brushes
 - Code and Resources
- Freehand Drawing, the Paintbrush Style
- Capturing the mouse
- Device Context, A Closer Look
- Displaying a Bitmap
- Animation at Work
 - WM_CREATE and *OnCreate()*
 - WM_TIMER and *OnTimer()*
 - A Few More Points...
- Windows, the Endless World...
- Summary
- Exercise

Since times immemorial colors and shapes have fascinated mankind like nothing else. Otherwise people would have still been using the character oriented interfaces of MS-DOS or Unix. In fact the graphical ability of Windows has played a very important role in its success story. Once you get a hang of how to draw inside a window it would open up immense possibilities that you never thought were possible.

Graphics as of Now

World has progressed much beyond 16 colors and 640 x 480 resolution graphics that Turbo C/C++ compilers offered under MS-DOS environment. Today we are living in a world of 1024 x 768 resolution offering 16.7 million colors. Graphical menus, icons, colored cursors, bitmaps, wave files and animations are the order of the day. So much so that a 16-color graphics program built using Turbo C working on a poor resolution almost hurts the eye. Moreover, with the whole lot of Windows API functions to support graphics activity there is so much that can be achieved in a graphics program under Windows. I am sure that this chapter will help you understand and appreciate these new capabilities.

Device Independent Drawing

Windows allow programmers to write programs to display text or graphics on the screen without concern over the specifics of the video hardware. A Windows program that works on a VGA display will work without modification on an SVGA or on a XGA display that Windows supports.

The key to this 'device independence' is Windows' use of a 'device context'. We will explore how the device context can be used for both text and graphics output, and how using the device context keeps our programs from interfering with each other on the screen.

During the original design of Windows, one of the goals was to provide ‘device independence’. Device independence means that the same program should be able to work using different screens, keyboards and printers without modification to the program. Windows takes care of the hardware, allowing the programmer to concentrate on the program itself. If you have ever had to update the code of an MS-DOS program for the latest printer, plotter, video display, or keyboard, you will recognize device independence as a huge advantage for the developer.

Windows programs do not send data directly to the screen or printer. A Windows program knows where (screen/printer) its output is being sent. However, it does not know how it would be sent there, neither does it need to bother to know this. This is because Windows uses a standard and consistent way to send the output to screen/printer. This standard way uses an entity called Device Context, or simply a DC. Different DC’s are associated with different devices. For example, a screen DC is associated with a screen, a printer DC is associated with a printer, etc. Any drawing that we do using the screen DC is directed to the screen. Similarly, any drawing done using the printer DC is directed to the printer. Thus, the only thing that changes from drawing to screen and drawing to printer is the DC that is used.

A windows program obtains a handle (ID value) for the screen or printer’s DC. The output data is sent to the screen/printer using its DC, and then Windows and the Device Driver for the device takes care of sending it to the real hardware. The advantage of using the DC is that the graphics and text commands that we send using the DC are always the same, regardless of where the physical output is showing up.

The part of Windows that converts the Windows graphics function calls to the actual commands sent to the hardware is the GDI, or Graphics Device Interface. The GDI is a program file called GDI32.DLL and is stored in the Windows System directory. The

Windows environment loads GDI32.DLL into memory when it is needed for graphical output. Windows also loads a ‘device driver’ program if the hardware conversions are not part of GDI32.DLL. Common examples are VGA.SYS for VGA video screen and HPPLC.SYS for the HP LaserJet printer. Drivers are just programs that assist the GDI in converting Windows graphics commands to hardware commands.

Thus GDI provides all the basic drawing functionality for Windows; the device context represents the device providing a layer of abstraction that insulates your applications from the trouble of drawing directly to the hardware. The GDI provides this insulation by calling the appropriate device driver in response to windows graphics function calls.

Hello Windows

We would begin our tryst with graphics programming under windows by displaying a message “Hello Windows” in different fonts. Note that though we are displaying text under Windows even text gets drawn graphically in the window. First take a look at the program given below before we set out to understand it.

```
# include <windows.h>
# include "helper.h"

void OnPaint ( HWND ) ;
void OnDestroy ( HWND ) ;

int __stdcall WinMain ( HINSTANCE hInstance, HINSTANCE hPrevInstance,
LPSTR lpszCmdline, int nCmdShow )
{
    MSG m ;

    /* Perform application initialization */
    InitInstance ( hInstance, nCmdShow, "Text" ) ;
```

```
/* Main message loop */
while ( GetMessage ( &m, NULL, 0, 0 ) )
    DispatchMessage(&m);

return 0 ;
}

LRESULT CALLBACK WndProc ( HWND hWnd, UINT message,
                           WPARAM wParam, LPARAM lParam )
{
    switch ( message )
    {
        case WM_DESTROY :
            OnDestroy ( hWnd ) ;
            break ;
        case WM_PAINT :
            OnPaint ( hWnd ) ;
            break ;
        default :
            return DefWindowProc ( hWnd, message, wParam, lParam ) ;
    }
    return 0 ;
}

void OnDestroy ( HWND hWnd )
{
    PostQuitMessage ( 0 ) ;
}

void OnPaint ( HWND hWnd )
{
    HDC hdc ;
    PAINTSTRUCT ps ;
    HFONT hfont ;
    LOGFONT f = { 0 } ;
    HGDIOBJ holdfont ;
    char *fonts[] = { "Arial", "Times New Roman", "Comic Sans MS" } ;
    int i ;
```

```
hdc = BeginPaint ( hWnd, &ps );

for ( i = 0 ; i < 3 ; i++ )
{
    strcpy ( f.lfFaceName, fonts[ i ] ); /* copy font name */
    f.lfHeight = 40 * ( i + 1 ); /* font height */
    f.lfItalic = 1 ; /* italic */

    hfont = CreateFontIndirect ( &f );
    holdfont = SelectObject ( hdc, hfont );

    SetTextColor ( hdc, RGB ( 0, 0, 255 ) );

    TextOut ( hdc, 10, 70 * i, "Hello Windows", 13 );

    SelectObject ( hdc, holdfont );
    DeleteObject ( hfont );
}

EndPaint ( hWnd, &ps );
}
```

On execution of this program the window shown in Figure 18.1 appears.



Figure 18.1

Drawing to a window involves handling the **WM_PAINT** message. This message is generated whenever the client area of the window needs to be redrawn. This redrawing would be required in the following situations:

- (a) When the Window is displayed for the first time.
- (b) When the window is minimized and then maximized.
- (c) When some portion of the window is overlapped by another window and the overlapped window is dismissed.
- (d) When the size of the window changes on stretching its boundaries.
- (e) When the window is dragged out of the screen and then brought back into the screen.

Would a **WM_PAINT** message be generated when the cursor is dragged in the window? No. In this case the window saves the area overlapped by the cursor and restores it when the cursor moves to another position.

When the **switch-case** structure inside **WndProc()** finds that the message ID passed to **WndProc()** is **WM_PAINT**, it calls the function **OnPaint()**. Within **OnPaint()** we have called the API function **BeginPaint()**. This function obtains a handle to the device context. Additionally it also fills the **PAINTSTRUCT** structure with information about the area of the window which needs to be repainted. Lastly it removes **WM_PAINT** from the message queue. After obtaining the device context handle, the control enters a loop.

Inside the loop we have displayed “Hello Windows” in three different fonts. Each time through the loop we have setup a **LOGFONT** structure **f**. This structure is used to indicate the font properties like font name, font height, italic or normal, etc. Note that in addition to these there are other font properties that may be setup. The properties that we have not setup in the loop are all initialized to **0**. Once the font properties have been setup we have called the **CreateFontIndirect()** API function to create the font.

This function loads the relevant font file. Then using the information in the font file and the font properties setup in the **LOGFONT** structure it creates a font in memory. **CreateFontIndirect()** returns the handle to the font created in memory. This handle is then passed to the **SelectObject()** API function to get the font into the DC. This function returns the handle to the existing font in the DC, which is preserved in **hfont** variable. Next we have used the **SetTextColor()** API function to set the color of the text to be displayed through **TextOut()**. The **RGB()** macro uses the red, green and blue component values to generate a 32-bit color value. Note that each color component can take a value from **0** to **255**. To **TextOut()** we have to pass the handle to the DC, position where the text is to be displayed, the text to be displayed and its length.

With **hfont** only one font can be associated at a time. Hence before associating another font with it we have deleted the existing font using the **DeleteObject()** API function. Once outside the loop we have called the **EndPaint()** API function to release the DC handle. If not released we would be wasting precious memory, because the device context structure would remain in memory but we would not be able access it.

In place of **TextOut()** we can also use the **DrawText()** API function. This function permits finer control over the way the text is displayed. You can explore this function on your own.

Drawing Shapes

If text is so near can graphics be far behind? Now that we know how to draw text in a window let us now create a simple program that displays different shapes in a window. Instead of showing the entire program given below is the listing of **OnPaint()**. The rest of the program is same as in the previous section. Here onwards I would be showing only the **OnPaint()** handler unless otherwise required.

```
void OnPaint ( HWND hWnd )
{
    HDC hdc ;
    PAINTSTRUCT ps ;
    HBRUSH hbr ;
    HGDIOBJ holdbr ;
    POINT pt[5] = { 250, 150, 250, 300, 300, 350, 400, 300, 320, 190 } ;

    hdc = BeginPaint ( hWnd, &ps ) ;

    hbr = CreateSolidBrush ( RGB ( 255, 0, 0 ) ) ;
    holdbr = SelectObject ( hdc, hbr ) ;

    MoveToEx ( hdc, 10, 10, NULL ) ;
    LineTo ( hdc, 200, 10 ) ;

    Rectangle ( hdc, 10, 20, 200, 100 ) ;

    RoundRect ( hdc, 10, 120, 200, 220, 20, 20 ) ;

    Ellipse ( hdc, 10, 240, 200, 340 ) ;

    Pie ( hdc, 250, 10, 350, 110, 350, 110, 350, 10 ) ;

    Polygon ( hdc, pt, 5 ) ;

    SelectObject ( hdc, holdbr ) ;
    DeleteObject ( hbr ) ;

    EndPaint ( hWnd, &ps ) ;
}
```

On execution of this program the window shown in Figure 18.2 appears.

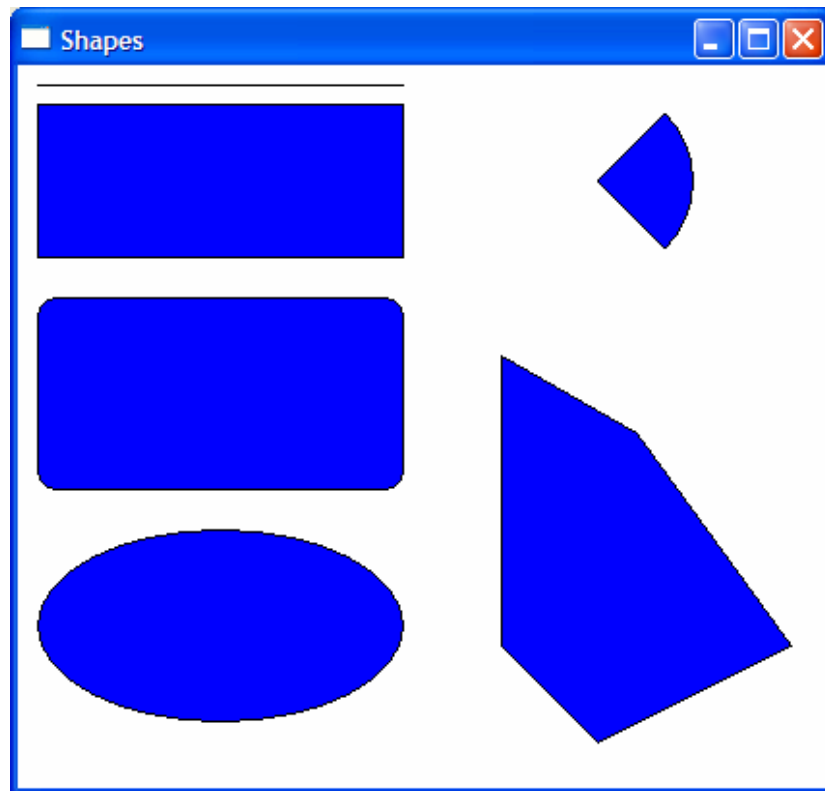


Figure 18.2

For drawing any shape we need a pen to draw its boundary and a brush to paint the area enclosed by it. The DC contains a default pen and brush. The default pen is a solid pen of black color and the default brush is white in color. In this program we have used the default pen and a blue colored solid brush for drawing the shapes.

As before, we begin by obtaining a handle to the DC using **BeginPaint()** function. For creating a solid colored brush we need to call the **CreateSolidBrush()** API function. The second parameter of this function specifies the color of the brush. The function returns the handle of the brush which we have preserved

in the **hbr** variable. Next we have selected this brush in the DC. The handle of the default brush in DC is collected in the **holdbr** variable.

Once we have selected the brush into the DC we are ready to draw the shapes. For drawing the line we have used **MoveToEx()** and **LineTo()** API functions. Similarly for drawing a rectangle we have used the **Rectangle()** function.

The **RoundRect()** function draws a rectangle with rounded corners. In **RoundRect (x1, y1, x2, y2, x3, y3)**, **x1, y1** represents the *x* and *y*-coordinates of the upper-left corner of the rectangle. Likewise, **x2, y2** represent coordinates of the bottom right corner of the rectangle. **x3, y3** specify the width and height of the ellipse used to draw the rounded corners.

Note that rectangle and the rounded rectangle are drawn from **x1, y1** up to **x2-1, y2-1**.

Parameters of **Ellipse()** specify coordinates of bounding rectangle of the ellipse.

The **Pie()** function draws a pie-shaped wedge by drawing an elliptical arc whose center and two endpoints are joined by lines. The center of the arc is the center of the bounding rectangle specified by **x1, y1** and **x2, y2**. In **Pie(x1, y1, x2, y2, x3, y3, x4, y4)**, **x1, y1** and **x2, y2** specify the *x* and *y*-coordinates of the upper left corner and bottom right corner respectively, of the bounding rectangle. **x3, y3** and **x4, y4** specify the *x* and *y*-coordinates of the arc's starting point and ending point respectively.

In **Polygon (lpPoints, nCount)**, **lpPoints** points to an array of points that specifies the vertices of the polygon. Each point in the array is a **POINT** structure. **nCount** specifies the number of vertices stored in the array. The system closes the polygon automatically, if necessary, by drawing a line from the last vertex to the first.

Once we are through with drawing the shapes the old brush is selected back in the DC and then the brush created by us is deleted using **DeleteObject()** function.

Types of Pens

In the previous program we have used the default solid black pen of thickness 1 pixel. We can create pens of different style, color and thickness to do our drawing. The following **OnPaint()** handler shows how this can be achieved.

```
void OnPaint ( HWND hWnd )
{
    HDC hdc ;
    PAINTSTRUCT ps ;
    HPEN hpen ;
    HGDIOBJ holdpen ;

    hdc = BeginPaint ( hWnd, &ps ) ;

    hpen = CreatePen ( PS_DASH, 1, RGB ( 255, 0, 0 ) ) ;
    holdpen = SelectObject ( hdc, hpen ) ;

    MoveToEx ( hdc, 10, 10, NULL ) ;
    LineTo ( hdc, 500, 10 ) ;

    SelectObject ( hdc, holdpen ) ;
    DeleteObject ( hpen ) ;

    hpen = CreatePen ( PS_DOT, 1, RGB ( 255, 0, 0 ) ) ;
    holdpen = SelectObject ( hdc, hpen ) ;

    MoveToEx ( hdc, 10, 60, NULL ) ;
    LineTo ( hdc, 500, 60 ) ;

    SelectObject ( hdc, holdpen ) ;
    DeleteObject ( hpen ) ;
}
```